

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Artificial Intelligence Application Development and Jira

Practical – II

Roll No.: T005	Name: Ritesh Chaubey
Class: TYIT	Batch: 1
Date of Assignment: 17/07/2026	Date/Time of Submission: 17/07/2026

AIM: Problem Solving by Searching (Uninformed Search).

- A. Given an initial configuration of the 8-puzzle and a goal configuration, write a Python program to find the shortest sequence of moves to reach the goal state using Breadth-First Search (BFS).**

CODE:

```
from collections import deque

# Function to display puzzle state
def print_state(state):
    print(f"{state[0]} {state[1]} {state[2]}")
    print(f"{state[3]} {state[4]} {state[5]}")
    print(f"{state[6]} {state[7]} {state[8]}")

# Function to find possible next states
def get_neighbors(state):
    neighbors = []

    # Find position of blank tile
    blank_index = state.index(0)

    # Possible moves
    moves = {
        "Up": -3,
        "Down": 3,
        "Left": -1,
        "Right": 1
    }

    for move, position_change in moves.items():
        new_index = blank_index + position_change

        # Check valid moves
        if move == "Up" and blank_index < 3:
            continue
        if move == "Down" and blank_index > 5:
            continue
        if move == "Left" and blank_index % 3 == 0:
            continue
        if move == "Right" and blank_index % 3 == 2:
```

```

        continue

    # Create new state by swapping blank tile
    new_state = list(state)
    new_state[blank_index], new_state[new_index] = (
        new_state[new_index],
        new_state[blank_index]
    )

    neighbors.append((tuple(new_state), move))

return neighbors

# BFS Function
def bfs(initial_state, goal_state):
    queue = deque()
    queue.append((initial_state, []))

    visited = set()
    visited.add(initial_state)

    while queue:
        current_state, path = queue.popleft()

        if current_state == goal_state:
            return path

        for neighbor, move in get_neighbors(current_state):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, path + [(move, neighbor)]))

    return None

# Main Program
initial_state = (
    1, 2, 3,
    4, 0, 6,
    7, 5, 8
)

goal_state = (
    1, 2, 3,
    4, 5, 6,
    7, 8, 0
)

print("INITIAL STATE")
print_state(initial_state)

print("\nGOAL STATE")
print_state(goal_state)

solution = bfs(initial_state, goal_state)

if solution:
    print("\n SHORTEST SEQUENCE OF MOVES")

```

```
for step, (move, state) in enumerate(solution, start=1):
    print(f"\nStep {step}: {move}")
    print_state(state)

print("Goal reached successfully!")
print("Total number of moves:", len(solution))
else:
    print("\nNo solution found.")
```

OUTPUT:

INITIAL STATE

1 2 3

4 0 6

7 5 8

GOAL STATE

1 2 3

4 5 6

7 8 0

SHORTEST SEQUENCE OF MOVES

Step 1: Down

1 2 3

4 5 6

7 0 8

Step 2: Right

1 2 3

4 5 6

7 8 0

Goal reached successfully!

Total number of moves: 2

- B. Given two water jugs of 4 litres and 3 litres capacity, write a Python program to obtain exactly 2 litres in one jug using Depth-First Search (DFS). [Vary capacity of jugs]**

CODE:

```
# Water Jug Problem using Depth-First Search (DFS)

def dfs(jug1, jug2, target):

    stack = [((0, 0), [])] # (current_state, path)

    visited = set()

    while stack:

        (x, y), path = stack.pop()

        if (x, y) in visited:

            continue

        visited.add((x, y))

        path = path + [(x, y)]

        # Goal check

        if x == target or y == target:

            return path

        next_states = [

            (jug1, y),          # Fill Jug 1

            (x, jug2),          # Fill Jug 2

            (0, y),             # Empty Jug 1

            (x, 0),             # Empty Jug 2

            (x - min(x, jug2 - y), y + min(x, jug2 - y)), # Pour Jug1 -> Jug2

            (x + min(y, jug1 - x), y - min(y, jug1 - x)) # Pour Jug2 -> Jug1

        ]

        for state in next_states:
```

```
        if state not in visited:

            stack.append((state, path))

        return None

# Main Program

jug1 = int(input("Enter capacity of Jug 1: "))

jug2 = int(input("Enter capacity of Jug 2: "))

target = int(input("Enter target amount: "))

solution = dfs(jug1, jug2, target)

if solution:

    print("\nGoal Achieved Successfully!")

    print("Solution Path:")

    for state in solution:

        print(state)

else:

    print("\nNo solution found.")
```

Output:-

```
Goal Achieved Successfully!
Solution Path:
(0, 0)
(0, 3)
(3, 0)
(3, 3)
(4, 2)
```

c) Given a weighted graph representing cities and distances between them, write a Python program to find the least-cost path from Arad to Bucharest using Uniform Cost Search (UCS). [Provide any other weighted graph for applying UCS].

Code:-

```
import heapq

graph = {
    'Mumbai': [('Pune', 150), ('Hyderabad', 710)],
    'Pune': [('Mumbai', 150), ('Bangalore', 840)],
    'Hyderabad': [('Mumbai', 710), ('Bangalore', 570)],
    'Bangalore': []
}

def ucs(start, goal):
    queue = [(0, start, [start])]
    visited = []

    while queue:
        cost, city, path = heapq.heappop(queue)

        if city == goal:
            return cost, path

        if city not in visited:
            visited.append(city)

            for neighbor, distance in graph[city]:
                heapq.heappush(
                    queue,
                    (
                        cost + distance,
                        neighbor,
                        path + [neighbor]
                    )
                )

    return None, None

cost, path = ucs("Mumbai", "Bangalore")
```

```
print("Least-cost path:")  
print("Path:", " -> ".join(path))  
print("Total Distance:", cost, "km")
```

Output:-

```
Least-cost path:  
Path: Mumbai -> Pune -> Bangalore  
Total Distance: 990 km
```